



ENGINEERING SPECIFICATION

PART NAME											PART NUMBER							
MODULE PROGRAMMING/CONFIGURATION DESIGN SPECIFICATION											DS-2L1T-1A322-AA							
LET																		
FR	1	1A	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
LET																		
FR	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	
DATE	LET	FR	REVISIONS								DR	CK	REFERENCE					
10/30/1998			INITIAL RELEASE										PREPARED/APPROVED BY:					
													CHECKED BY:					
													DETAILED BY:					
													CONCURRENCE / APPROVAL SIGNATURES					
													DESIGN ENGINEERING SUPERVISOR					
													DESIGN ENGINEERING					
													MANUFACTURING ENGRG					
													QUALITY CONTROL					
													PURCHASING					
													SUPPLIER QUALITY ASSISTANCE					
FRAME 1 OF 33																		

ENGINEERING SPECIFICATION

PART NAME											SPECIFICATION - ELECTRICAL CONFIGURATION											PART NUMBER							
MODULE PROGRAMMING/CONFIGURATION DESIGN SPECIFICATION											DS-2L1T-1A322-AA																		
LET																													
FR	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50												
LET																													
FR	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67												
LET																													
FR	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83													
LET																													
FR																													
DATE		LET	FR	REVISIONS							DR	CK	REFERENCE																
													PREPARED/APPROVED BY:																
													CHECKED BY:																
													DETAILED BY:																
													CONCURRENCE / APPROVAL SIGNATURES																
													PRINCIPAL DESIGN ENGINEERING																
													DESIGN ENGINEERING MGT.																
													PEO MATERIAL ENGRG																
													PEO RELIABILITY ENGRG																
													MANUFACTURING ENGRG																
													QUALITY CONTROL/SQA																
													PURCHASING																

FRAME

2 OF 33



MODULE PROGRAMMING/ CONFIGURATION DESIGN SPECIFICATION

Worldwide Requirements



*Advanced Vehicle Technology
Electrical/Electronic Systems Engineering
Advance & Core Diagnostics*

DS-2L1T-1A322-AA

October 30, 1998

FOREWORD

This document contains information that is proprietary to the Ford Motor Company. Recipients of this document shall not duplicate, use or disclose information contained herein, in whole or in part, for other than company purposes.

Distribution	Date Issued	Ver.	WERS Alert:	Date Issued	Written By:	PROFS	Prepared By:
Rev A	5/14/97	A	NB00-I-10661251-303	May. 14, 1997	George Kawwas x08538	GKAWWAS	GKAWWAS
Rev A (new)	1030/98	A(new)			Steve DiLodovico xx88815	SDILODOV	SDILODOV

PLEASE READ

This is a dynamic document within FORD Motor Company that will be periodically updated as required to make recommended and/or necessary improvements. Any recommendations for improving this document would be greatly appreciated. Please forward any and all suggestions, comments, and/or questions regarding this specification, via PROFs, to the author(s) listed for the most recent revision date.

1. INTRODUCTION	6
1.1 PURPOSE.....	6
1.2 VISION:.....	6
1.2.1 <i>Vehicle Information Back Up</i>	7
2. MODULE DESIGN AND OTHER NEEDED ELEMENTS FOR PROGRAMMING AND CONFIGURATION	13
2.1 ACCEPTABLE DATA FILE FORMATS	13
2.1.1 <i>Header Section (Applies to both Ford defined and Intel Hex-32 bit Code 99 file formats)</i>	13
2.1.2 <i>Ford Powertrain Intel Hex 32-bit Code 99 File Header Example</i>	14
2.1.3 <i>Intel 32-bit Code 99 File Format Example (Non - Ford Powertrain example)</i>	14
2.1.4 <i>Ford Defined File Format</i>	16
2.2 DATA TRANSFER METHODS	17
2.2.1 <i>Method 2 Write Memory Block</i>	17
2.2.2 <i>Information Pertaining to both Methods 3 and 4</i>	18
2.2.3 <i>Method 3 Download Block</i>	18
2.2.4 <i>Method 4 FLASH Download</i>	23
2.3 UNSUPPORTED METHOD 1 CONFIGURATION	28
2.4 BASE STRATEGY.....	28
2.5 PROGRAMMING ERROR DETECTION, VERIFICATION AND VISUAL INDICATORS	28
2.6 INFORMATION BACK-UP	29
3. SERVICE REQUIREMENTS	30
3.1 OVERVIEW.....	30
3.2 SERVICE PROCEDURES FOR MODULE PROGRAMMING/CONFIGURATION	30
3.2.1 <i>Inhale/Exhale Method of Module Configuration</i>	30
3.2.2 <i>On-line database access</i>	32
4. REFERENCES	33
4.1 FORD SPECIFICATIONS.....	33
4.2 SAE SPECIFICATIONS.....	33

1. INTRODUCTION

1.1 PURPOSE

Electronic Control Units (ECUs or electronic modules) have demonstrated continued increases in feature count, memory size, I/O circuit count and part number complexity. All areas of the vehicle electrical/electronic system are affected, particularly powertrain (EEC-V and PTEC) and body computers (GEM, LCM, DDM, DSM, etc.). Electronic Control Units are experiencing late and frequent changes, many of which affect software. These trends will undoubtedly continue into the foreseeable future.

There is a need to reduce vehicle build and service complexity, reduce ECU supplier complexity, and allow for maximum flexibility in software design by establishing designs and procedures providing the ability to configure and program ECUs in assembly and in service. Configuration and module programming will allow Module Design and Release activities to minimize part complexity, economies of scale and allow bookshelf software to be utilized.

The scope of this document is to define the process and software requirement for the implementation and support of module programming and configuration for service and assembly.

1.2 VISION:

Allow any microprocessor-based ECU to be configured or fully programmed (up to and including 100% of the module's code) at any assembly plant or dealership, via the Ford approved data links.

The following requirements/guiding principles shall be met for module programming/configuration:

1. The ECU shall be on a Ford approved data communication link and accessible from the Diagnostic Link Connector (DLC).
2. The ECU shall either have EEPROM or Flash Memory for configuration.
3. Each assembly plant and service facility implementing module configuration/programming will use standard equipment approved by VO and FCSD.
4. ECUs are to be programmed/configured fully at End-Of-Line (EOL), with the goal of adding minimal cycle time to the assembly process.
5. The act of module configuration/programming will follow the guidelines set forth by this specification.
6. Module configuration is intended for the adding, deleting, or modifying of module behavior which are required to operate differently due to legal or local market requirements, vehicle variables (i.e., tire size, axle ratio) or marketing/sales strategies.
7. Backup of configuration in VID.

1.2.1 Vehicle Information Back Up

Table 2-1 in 1.2.1.1 contains the generic information that is to be included in the VID block. It consists of the information such as vehicle VIN, feature/function content (e.g., Tire-axle ratio, power windows, etc.), download file extension and the modules equipped on the vehicle. Feature/function content is intended to be a back-up for individually programmed modules. Service tools will be able to retrieve a module's configuration from the PCM and reconfigure the replaced module; this backup will reduce service time as well as minimizing information needed from customers on which options their vehicles were equipped with.

Application specific VID block definition resides in the Vehicle Specific Configuration Specification (VSCS) and contains all possible configuration settings for a particular vehicle line.

NOTE: VID default condition is 0xFF in all locations except for the last byte that indicates the ECU type.

1.2.1.1 Generic Vehicle ID Block Configuration

Byte #	Parameter	Comments	Description
1--21	VIN	e.g. "1MEPM6046KH693300FFFF"	Vehicle Identification Number (VIN)
22	Delimiter	"*" (0x2A)	
23	Octane Adjust/Spark Retard	e.g. lookup "A" (0x41)	Used in service to replace Octane Adjust Pin previous available to allow technician to request additional spark retard.
24	Octane of Fuel	e.g. lookup "B" (0x42)	Used to change spark retard from 95 RON base to 91 RON.
25	Fuel Type	e.g. lookup "C" (0x43)	Used to change fuel stoichiometry for markets with Gasoline fuel.
26	Vehicle Speed Sensor (VSS) Limit	0 - 255 kph (0x00 - 0xFF)	Currently not defined.
27-28	Tire Size	revs / mile in hex (high byte, low byte for PTEC, low byte, high byte for EEC V)	Used to generate vehicle speed from OSS sensor.
29-30	Axle Ratio	= 1024 x ratio	Used to generate vehicle speed from OSS sensor.
31	Market Segment/Emissions	TBD	Currently not defined.
32-b0	Speed Control Installed	0 = present, 1 = not present	Used to indicate PCM integrated Speed Control is present.
32-b1	Four Wheel Drive	0 = 4x4, 1 = 4x2	Used to select if 4x4L input is a valid input.
32-b2	Manual Shift on the Fly	0 = Manual, 1 = Not Manual	Used to indicate if PCM should use TCSS instead of OSS for vehicle speed calculation.
32-b3-40	Future Powertrain Items	TBD	
41	Node ID for config. block # 1	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
42	Node ID for config. block # 2	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
43	Node ID for config. block # 3	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
44	Node ID for config. block # 4	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
45	Node ID for config. block # 5	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
46	Node ID for config. block # 6	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
47	Node ID for config. block # 7	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
48	Node ID for config. block # 8	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
49	Node ID for config. block # 9	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
50	Node ID for config. block # 10	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Configurable ECU
51-55	Configuration data or file ext. for block #1	unused byte(s) = 0x00, unused block(s) = 0xFF	Hex Data for Configurable ECU in Byte 41
56-60	Configuration data or file ext. for block #2	unused byte(s) = 0x00, unused block(s) = 0xFF	Hex Data for Configurable ECU in Byte 42

61-65	Configuration data or file ext. for block #3	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 43
66-70	Configuration data or file ext. for block #4	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 44
71-75	Configuration data or file ext. for block #5	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 45
76-80	Configuration data or file ext. for block #6	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 46
81-85	Configuration data or file ext. for block #7	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 47
86-90	Configuration data or file ext. for block #8	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 48
91-95	Configuration data or file ext. for block #9	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 49
96-100	Configuration data or file ext. for block #10	unused byte(s) = 0x00, unused block(s) = 00xFF	Hex Data for Configurable ECU in Byte 50
101-114	Node IDs for non-config. mods. 1-14	0x00-0xFE = present, 0xFF = not present	Hex Module ID for Non-Configurable ECUs.
115	Destination Country Byte1	Broadcast Generated Destination Country	Country Code for Restricting Configuration
116	Destination Country Byte2		in regulated markets.
117-122	Service Tool Checksum Equalizers	(FACTS = 0xFF's)	Allows VID changes without complete re-flash of PCM.
123	Future Service Tool Items	(FACTS = 0xFF)	Reserved for future use
124	Service Tool Data Integrity	0x00-0xFF (FACTS = 0xFF, NGS = 0xF2, etc.)	Which tool last changed the VID and the integrity of the data.
125-126	Vehicle ID block checksum	0x0000-0xFFFFE	Provides a check for validating the data.
127	VID block configuration identifier	(FORD = 0xFF, JAGUAR = 0xFE)	Determines the format of the VID.
128	IVFER use	0x00 = 512 PTEC, 0x01 = 1024 Intel, 0x10 = 256 Puma	Indicates target controller and size.

Table 2-1 Generic VID Block Template

1.2.1.2 Methods 1, 2, 3, and 4 Arrangements in VID Block

Each node ID and corresponding configuration data is to be arranged in a specific fashion. For instance, if a module implements Methods 1, 2 and 3 for configuration, the Method 1 node ID is to occur before the Method 2 node ID and the Method 2 node ID is to occur before the Method 3 node ID in the VID block. For a module populating more than one configuration block in the VID for Method 2, they are to be arranged such that the first Method 2 configuration data set is to have block 0 associated with it and each additional Method 2 configuration data set block number is to have the previous configuration data set block number incremented by one.

Methods 3 file extensions are to be stored in the VID block. A unique identifier to distinguish that a file extension resides in the 5th byte of the block is to be used. \$F0 \$F0 \$F0 \$F0 is the identifier to be placed in the first four bytes of a data block that contains a file extension, see Example 1-1.

The following example details the setup of a module that supports Methods 1, 2 and 3. Two blocks of data are associated with Method 2 in this example.

VID Block Byte #	Parameter	Comments	Description
41	\$28	Corresponds to data in bytes 51-55	ABS node ID (Method 1)
42	\$28	Corresponds to data in bytes 56-60	ABS node ID (Method 2 block 0)
43	\$28	Corresponds to data in bytes 61-65	ABS node ID (Method 2 block 1)
44	\$28	Corresponds to data in bytes 66-70	ABS node ID (Method 3)
51	\$xx	Method 1 data before Method 2 data	Method 1 data byte 1
52	\$xx	Method 1 data before Method 2 data	Method 1 data byte 2
53	\$xx	Method 1 data before Method 2 data	Method 1 data byte 3
54	\$xx	Method 1 data before Method 2 data	Method 1 data byte 4
55	\$00	Not used for Method 1	Placeholder for Method 1 unused byte
56	\$xx	Method 2 block 0 before block 1	Method 2 block 0 data byte 1
57	\$xx	Method 2 block 0 before block 1	Method 2 block 0 data byte 2
58	\$xx	Method 2 block 0 before block 1	Method 2 block 0 data byte 3
59	\$xx	Method 2 block 0 before block 1	Method 2 block 0 data byte 4
60	\$xx	Method 2 block 0 before block 1	Method 2 block 0 data byte 5
61	\$xx	Method 2 block 1 after block 0	Method 2 block 1 data byte 1
62	\$xx	Method 2 block 1 after block 0	Method 2 block 1 data byte 2
63	\$xx	Method 2 block 1 after block 0	Method 2 block 1 data byte 3
64	\$xx	Method 2 block 1 after block 0	Method 2 block 1 data byte 4
65	\$xx	Method 2 block 1 after block 0	Method 2 block 1 data byte 5
66	\$F0	First byte to identify file index	First file index identifier
67	\$F0	Second byte to identify file index	Second file index identifier
68	\$F0	Third byte to identify file index	Third file index identifier
69	\$F0	Forth byte to identify file index	Forth file index identifier
70	\$xx	Specifies the file extension for the Method 3 file	Method 3 file extension

Example 1-1 Example VID Block

1.2.1.3 Recommended Tester Method Detection Strategy

In order for a tester to read information out of the VID block and program a module, it must know what method to use in order to electronically write to a module. Fig 4-0 is the recommended strategy to detect what method is associated with each piece of VID block information.

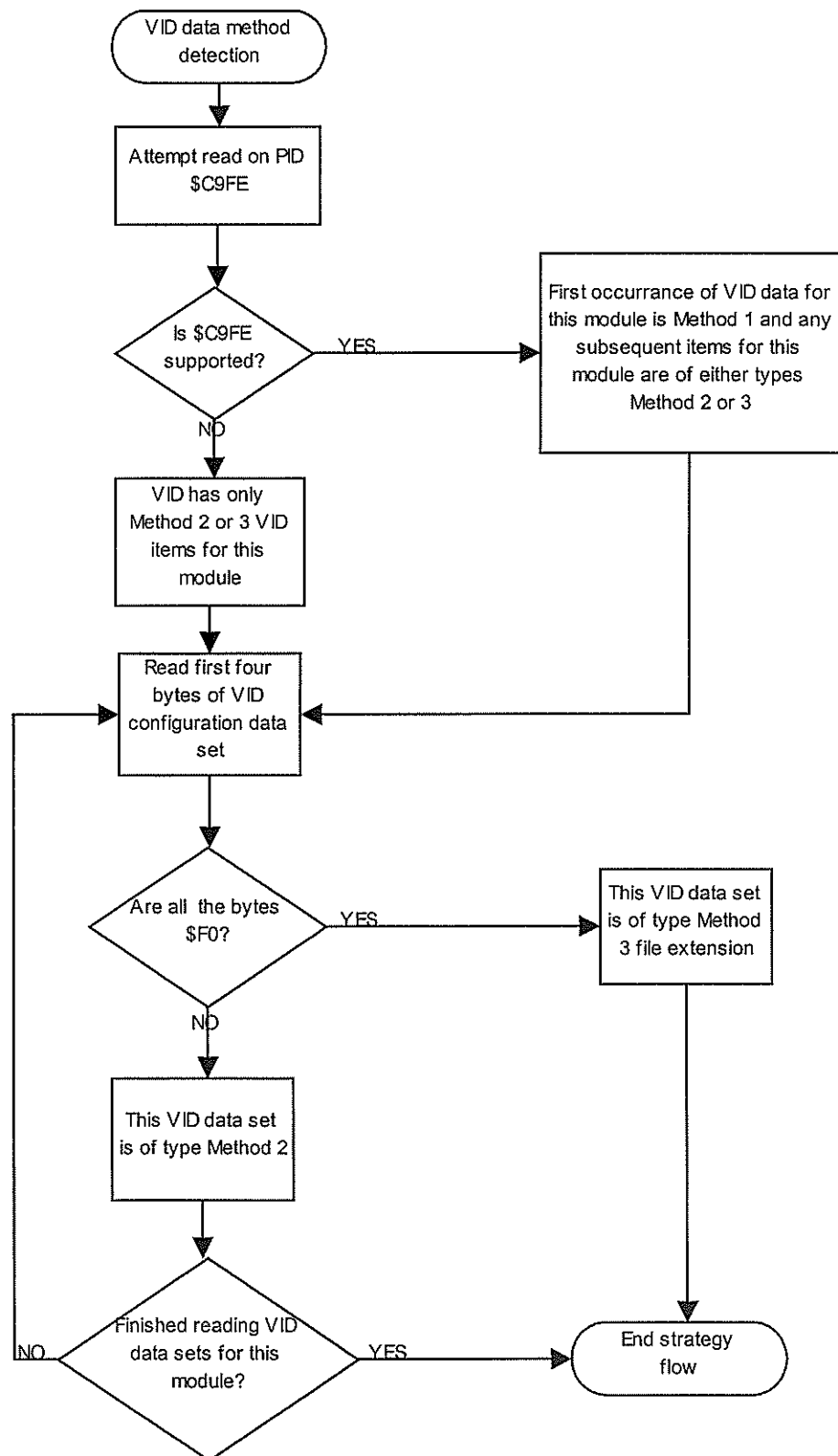


Figure 4-0 VID Block Method Detection Strategy

2. Module Design and Other Needed Elements for Programming and Configuration

2.1 Acceptable Data File Formats

2.1.1 Header Section (Applies to both Ford defined and Intel Hex-32 bit Code 99 file formats)

NOTE: This header applies to all data files for all modules except for Ford Powertrain modules.

The Header Section will consist of a list of fields, each on a separate line. The Header Section will end with an ASCII '\$' (0x24) on the line following the last field.

Field format:

Each field will consist of a name-value pair, ending with an ASCII Carriage Return and Line Feed. The format is as follows:

[field name]>[field value][Carriage Return][Line Feed]

Where:

[field name] is 1 to 80 ASCII characters (and can include a [Carriage Return] and/or [Line Feed].

> is the ASCII character for the 'greater than sign' (0x3E).

[field value] is 1 to 80 ASCII characters, starting at the first printable character after the 'greater than sign'.

[Carriage Return] is an ASCII Carriage Return (0x0D).

[Line Feed] is an ASCII Line Feed (0x0A).

NOTE: Field names must appear in the Header Section in the order shown below and all fields are mandatory. Any field names that do not apply to a system will be have definitions left blank. Hex data is to be represented with the prefix identifier 0x (hex value). The mandatory field names for the download file are:

<u>Field Name</u>	<u>Definition</u>
APPLICATION	Usage for programming file (e.g. DEW98 (Option X) interior audio file equalizer curve)
MASK NUMBER	Software Version Number as defined by PID 0xE200
FILE NAME	Name of programming file
RELEASE DATE	Date of file release
MODULE TYPE	Specific module application(s)
PRODUCTION MODULE PART NUMBER	Ford engineering part number
WERS NOTICE	WERS release number
CAL ID	Module calibration identification
COMMENTS	File description and/or usage
RELEASED BY	Individual releasing programming information (name, phone, etc.)
MODULE NAME	Name of module to be programmed
MODULE ID	Hexadecimal address of module needing programming
BLOCK COUNT	Number of blocks contained in the data file
BYTE COUNT	The number of data bytes for each block of data (i.e. 5,12,54 meaning there is 5 data bytes in block 1 and 54 bytes in block 3)

DOWNLOAD FORMAT	This determines the characteristics of the data transfer (i.e. handshaking or not)
FILE CHECKSUM	Checksum pertaining to all of the data in the file. Only data that is to be downloaded to the module is summed here, no block checksums are to be used in this calculation.
NUM OF ADR BYTES	Used to determine how many bytes used to address the module memory. It is a logical address.
FILE FORMAT	0 = Intel Hex 32-bit Code 99, 1 = Ford Defined Format

2.1.2 Ford Powertrain Intel Hex 32-bit Code 99 File Header Example

```

EPROM PART NO.          > 1M ;1234567890A
COPYRIGHT CALENDER YR > 1997
APPLICATION              > TEST FILE
FILE TYPE                > PTEC
FILE NAME                > OUT.HEX
RELEASE DATE            > Wed Dec 4 12:09:55 1996
MODULE TYPE              > PTEC-XX
VEHICLE CALIBRATION      > VCAL
VEHICLE APPLICATION      > VAPPL
ENGINE SIZE              > 12
PRODUCTION MODULE
    PART NUMBER > PTEC123
CATCHWORD                > CAT
WERS NOTICE              > N/A
DESIGN TRANSMITTAL        > N/A
CAL ID                   > ;2000
LAST DATA ADDRESS        > 000838DB
COMMENTS > FOR PRODUCTION USE ONLY

```

```

RELEASED BY > LINDA HODGSON    DIV/DEPT > ELD/PECE  TEL > 376656

```

2.1.3 Intel 32-bit Code 99 File Format Example (Non - Ford Powertrain example)

A 2-byte checksum is to reside in the last two data bytes of each block to be downloaded. It is to be calculated as the sum of all of the data to be downloaded to the module for that block.

The following is a theoretical example of an Intel-32 bit configuration data file:

```

APPLICATION>Expedition with RCM
MASK NUMBER>7 OR LATER
FILE NAME>XL1_RCM.1
RELEASE DATE>03/26/98
MODULE TYPE>Restraint Control Module
PRODUCTION MODULE PART NUMBER>XL5A-14B321-AA
WERS NOTICE>DE00E10757919001
CAL ID>NOT APPLICABLE
COMMENTS>
RELEASED BY>D&R engineer name and number
MODULE NAME>RESTRAINTS CONTROL MODULE
MODULE ID>0x58
BLOCK COUNT>1
BYTE COUNT>16
DOWNLOAD FORMAT>0x81
FILE CHECKSUM>0x0995
NUM OF ADR BYTES>4
FILE FORMAT>0
$

```

```

:020000040000FA
:0200000200807C
:1000A0001E9650016CDC4FFFFFFFFFFFFFFFFF099528
:00000001FF

```

NOTE: The bolded \$0995 is the 2-byte block checksum for this data block to be downloaded to the module. The file checksum is the same as the block checksum for this example because there is only one block to download to the module.

The following is a detailed explanation of each line of the data section Implementing Intel Hex-32 Format. It was developed by referencing the 2900 User Manual, Translation Formats, Intel Hex-32, Code 99, pages 7-46 and 7-47.

Line 1, :020000040000FA

: = start character

02 = number of data bytes

0000 = address

04 = Record Type

00 = Data Record

01 = End Record

02 = Extended Segment Address Record

04 = Extended Linear Address Record

06 = Block Separator Record

0000 = Offset address

FA = 2's compliment of this record's checksum

Calculate Checksum

02

+ 00

+ 00

+ 04

+ 00

+ 00

06

Then, determine it's 2's compliment:

0000 0110 = 06

Invert result:

1111 1001 = F9

Add 1:

1111 1001

+ 1

1111 1010 = FA

Line 2, :0200000200807C

: = start character

02 = number of data bytes

0000 = address

02 = Extended Segment Address Record

0080 = Offset address

7C = 2's compliment of this record's checksum

Line 3, :1000A0001E9650016CDC4FFFFFFFFFFFFFFFFF099528

: = start character

10 = number of data bytes

00A0 = address

00 = Data Record

1E9650016CDC4FFFFFFFFFFFFFFFFF0995 = Data

28 = 2's compliment of this record's checksum

Line 4, :00000001FF

: = start character

00 = number of data bytes

0000 = address

01 = End Record

FF = 2's compliment of this record's checksum

2.1.3.1 Block Separator:

The **Block Separator** record (06) shown in the following example is used to separate blocks of data that are to be sent to the module from a tester. A **Block Separator** is to be placed between data records in positions where it is intended to begin transmission of a new download sequence by the tester and is set up by a new **Download Routine Entry** (mode \$34) message. A block separator isn't to have any data associated with it.

The following file fragment contains an example data portion of a Hex 32-bit file without the header:

```
:020000040000FA
:0200000200807C
:1000A0001E9650016CDC4FFFFFFFFFFFFFFFF099528
:000000006FA
:0900E0001E9650016CDC4F029CDD
:00000001FF
```

The bolded 06 is the record type for a block separator and is placed between two data records that are to be downloaded in two separate blocks by the tester.

2.1.4 Ford Defined File Format

The Ford File Format has been defined to better facilitate creating configuration/programming data files by hand. Code 99 data files are often created from program code object files that are fed through a translator. Code 99 is difficult to create by hand which is why the Ford Defined Format was introduced.

A structure to match the needed elements of the diagnostic services for Methods 3 and 4 is the intention of Ford Defined Format. It was set up to contain only the information that the tester needs to perform necessary downloads to a module and make the user input structure of that data simple and straight forward.

Only Intel Hex 32-bit formatted files will be sent to FACTS and the service community; therefore, a translation utility will be produced to produce Hex 32-bit files from the Ford defined file format. Estimated time of delivery for the utility isn't known at this time.

The following is a theoretical example of a configuration data file:

```
APPLICATION>Expedition with RCM
MASK NUMBER>7 OR LATER
FILE NAME>XL1_RCM.1
RELEASE DATE>03/26/98
MODULE TYPE>Restraint Control Module
PRODUCTION MODULE PART NUMBER>XL5A-14B321-AA
WERS NOTICE>DE00E10757919001
CAL ID>NOT APPLICABLE
COMMENTS>This is a Ford defined file format example
RELEASED BY>D&R engineer name and number
MODULE NAME>RESTRAINTS CONTROL MODULE
MODULE ID>0x58
```

BLOCK COUNT>2
BYTE COUNT>15,9
DOWNLOAD FORMAT>0x81
FILE CHECKSUM>0B66
NUM OF ADR BYTES>3
FILE FORMAT>1
\$

BLOCK>1
NUMBER OF DATA BYTES>15
LOGICAL ADDRESS>0x80010A
CHECKSUM>0x0674
0x28 0x11 0xC4 0x45 0xE1 0x8F 0xA7 0x19 0x55 0x78 0x53 0x10 0xAC 0x8A 0x9C

BLOCK>2
NUMBER OF DATA BYTES>9
LOGICAL ADDRESS>0xA12C73
CHECKSUM>0x04F2
0xF4 0x67 0xB7 0xA2 0x38 0x88 0xCE 0x25 0x8B
(End of File)

2.2 Data Transfer Methods

The allowable programming methods available to configure or program modules in assembly or service are defined in this section. The three methods described below are the only methods that have been identified as acceptable and the use of any other methods will not be supported. Security algorithm direction is to be followed as specified in the Generic Diagnostic Specification (Part 1) and will be listed in a module's Part 2 specification whether it supports it or not. See section 2.5 for download verification strategies.

NOTE: A module is to disable any diagnostic feature that is turned off using configuration (i.e. logging DTCs and updating PIDs). A module is to dynamically turn diagnostic functionality on and off to correlate with each configurable function having diagnostics associated with it.

2.2.1 Method 2 Write Memory Block

The **Write Memory Block** (mode \$3B) diagnostic service, used for configuration/programming, will allow up to 5 bytes of data to be written to an ECU for each message transfer. Method 2 is intended to allow an off-board tester to modify relatively small amounts of memory in comparison with Method 3. The particular blocks of memory which each ECU will allow to be modified shall be known by the tester. The data can be formatted in any of the data types listed in the latest version of the Part 1 Global Diagnostic Specification. Also, all information downloaded with Method 2 will be specified in a corresponding module Part II Spec.

The response to the **Write Memory Block** message is the **General Response** (mode \$7F) message with the appropriate response code. Refer to the Part 1 Generic Diagnostic Specification for a listing of the response codes.

All blocks used for this method of configuration transfer will be of the logical type, no physical addressing will be used.

2.2.1.1 Scenario when a module has more than one logical block to address

In the case where there are multiple Method 2 blocks existent for a module, the first Method 2 occurrence is to have logical block zero associated with it. Each additional Method 2 block will have the logical address of the prior block incremented by one.

2.2.1.2 Read Block

Read Memory Block (mode \$3C) is to be used to read or verify any mode \$3B write. The block numbers used with \$3B map directly one-to-one with mode \$3C; therefore, any configuration/program written to memory with a mode \$3B can be read and verified with a mode \$3C using the same logical block number. The update of internal module memory prior to verification will not require a power cycle through the ignition key; however, the module may need to transition to operational state before re-entering diagnostic state to perform mode \$3C. A transition to operational state may be necessary to perform a re-initialization on some modules.

2.2.2 Information Pertaining to both Methods 3 and 4

For non-Ford defined powertrain modules, a two byte checksum is to be provided for each block of data and will be located in the last two bytes of the last data block message. A data block is defined to be the set of data to be transferred as defined in the mode \$34 setup of "Request Download Routine Entry".

Information to be downloaded utilizing Methods 3 and 4 will reside in a data file of one of the two defined file formats mentioned above. A two byte file checksum will be provided in the file header that will be the sum of all of the data provided in the data file; the data consists of all of the data to be downloaded into the module memory.

2.2.2.1 File Naming and Storage

File naming convention for Method 3/4 download files are to parallel the Part 2 naming convention with a difference in the extension as listed in Table 4-1.

Char 1	Char 2	Char 3	Char 4	Char 5	Char 6	Char 7	Char 8	Ext 1	Ext 2	Ext 3
PRE 1	PRE 2	PRE 3	_____	ACR 1	ACR 2	ACR 3	ACR 4	NUM 1	NUM 2	NUM 3

Table 4-1 Methods 3 and 4 Data File Naming Convention

PRE 1, PRE 2 and PRE 3 are the first 3 characters of the WERS prefix part number for the module requiring the configuration data file. ACR 1 through ACR 4 are the characters of the established module acronym, note that ACR 4 may or may not exist for every module. If ACR 4 doesn't exist it is to not be used in the filename. NUM 1 through NUM 3 are sequential numbers assigned to each successive data file defined for a module. Char 4 is always an underscore to separate the prefix from the acronym.

NOTE: The file extension for Methods 3 and 4 cannot exceed decimal 256; this is because the VID storage for this item is only one byte in size.

For non-Ford defined powertrain modules, all Method 3 and 4 data files are to be stored in a "data" sub-directory residing on an AVT server. Any activity requiring these files is to retrieve them from the AVT storage location (see appropriate WWDiag diagnostic application engineer for assistance).

2.2.3 Method 3 Download Block

Download block method provides a means for an off-board tester to send multiple bytes of data, up to 1 megabyte for 3 byte addressing and 4 kilobytes for 4 byte addressing, to an ECU. Message sequencing consists of a means for initiating the download, transfer of data messages and exit of the download operation as detailed in figure 4-1.

Byte 5 of the **Request Download Routine Entry** (mode \$34) service contains format information as how the data transfer is to occur and is detailed in Table 4-2. Address size defines whether 24 or 32-bit addressing is used. If

32-bit addressing is used, then data byte 4 is the most significant address byte of the mode \$34 message; otherwise, byte 4 it is the least significant byte of the block size. The amount of bytes to download is determined by multiplying the Byte Count Multiplier field by the byte count defined in the mode \$34 service and detailed in the Part 1 specification. Handshaking is performed with an affirmative general response message as detailed in Fig 4-1. Additional information on diagnostic services can be found in the "Global Diagnostic Specification - Part 1".

Address Size	Byte Count Multiplier	Reserved	Handshaking Identifier
Bit 7	Bits 6, 5	Bits 4, 3, 2, 1	Bit 0
1 = 3 byte pointer 0 = 4 byte pointer	00 = Multiply by 1 01 = Multiply by 4 10 = Multiply by 8 11 = Multiply by 16	Reserved	0 = No handshake 1 = Handshake

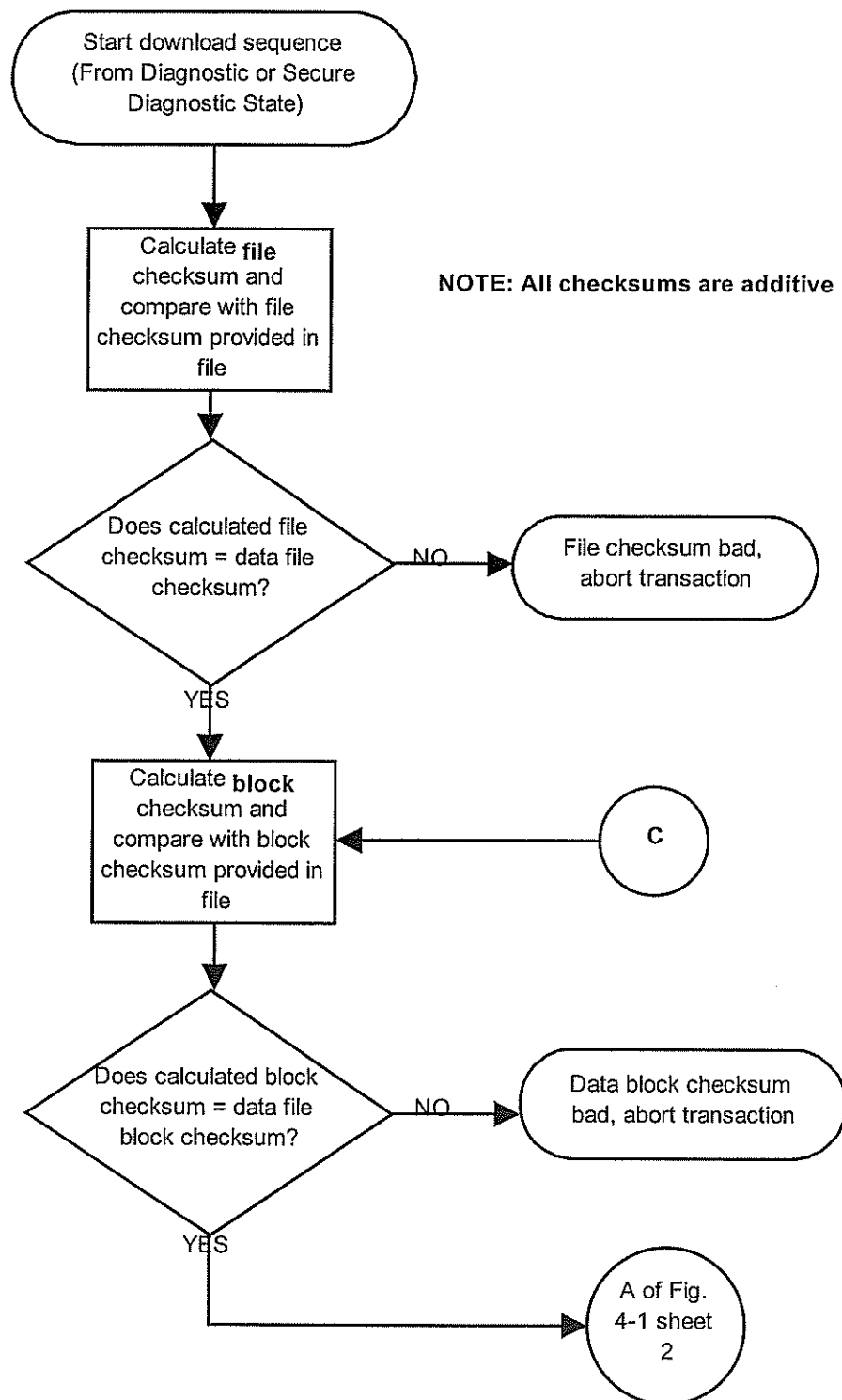
Table 4-2 Block Transfer Format Byte Encoding

Table 4-2 is backward compatible with old versions of the Part 1 specification. Ford Powertrain uses the version of table 4-2 as specified in an obsolete specification, the SCP-002 "Diagnostic Message and Dialogue Requirements" part number **ES-F7LC-12K529-CB**. Powertrain committed to attempt to transition over to Table 4-2 when feasible.

Addressing Method

Method 3 download uses logical addressing for the block start address. All logical block addresses are to be detailed in the module specific Sub-System Diagnostic Specification (Part II).

NOTE: Only certain error conditions are provided in the accompanying flowcharts (Figs. 4-1 and 4-2), please refer to the Part 1 specification for additional error handling. Also, any module or data security requirements a module may have are assumed to be taken care of before entering Figs. 4-1 and 4-2.

(TESTER)**Figure 4-1 Method 3 Download Procedure with and without handshake (Sheet 1 of 3)**

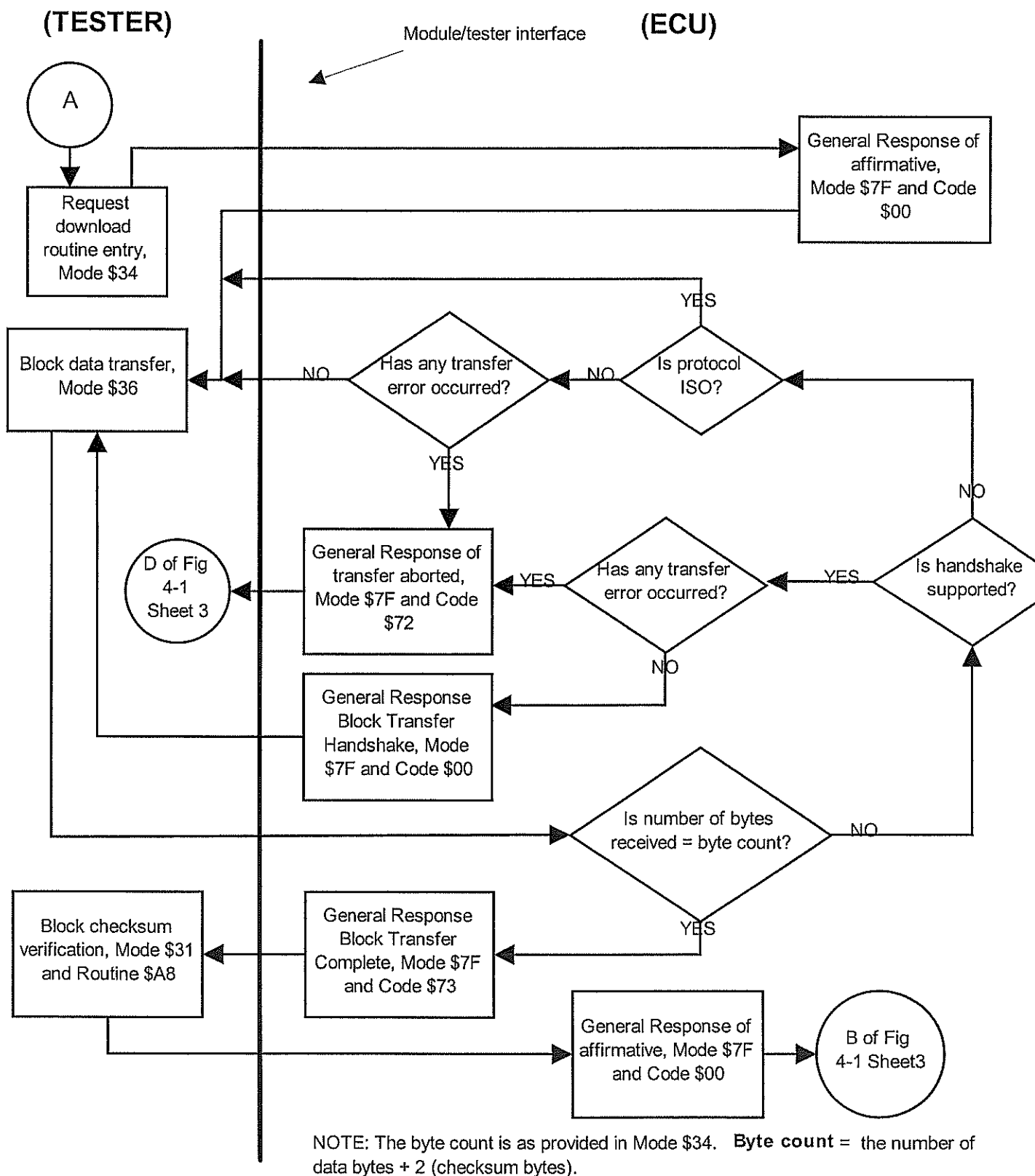


Figure 4-1 Method 3 Download Procedure with and without handshake (Sheet 2 of 3)

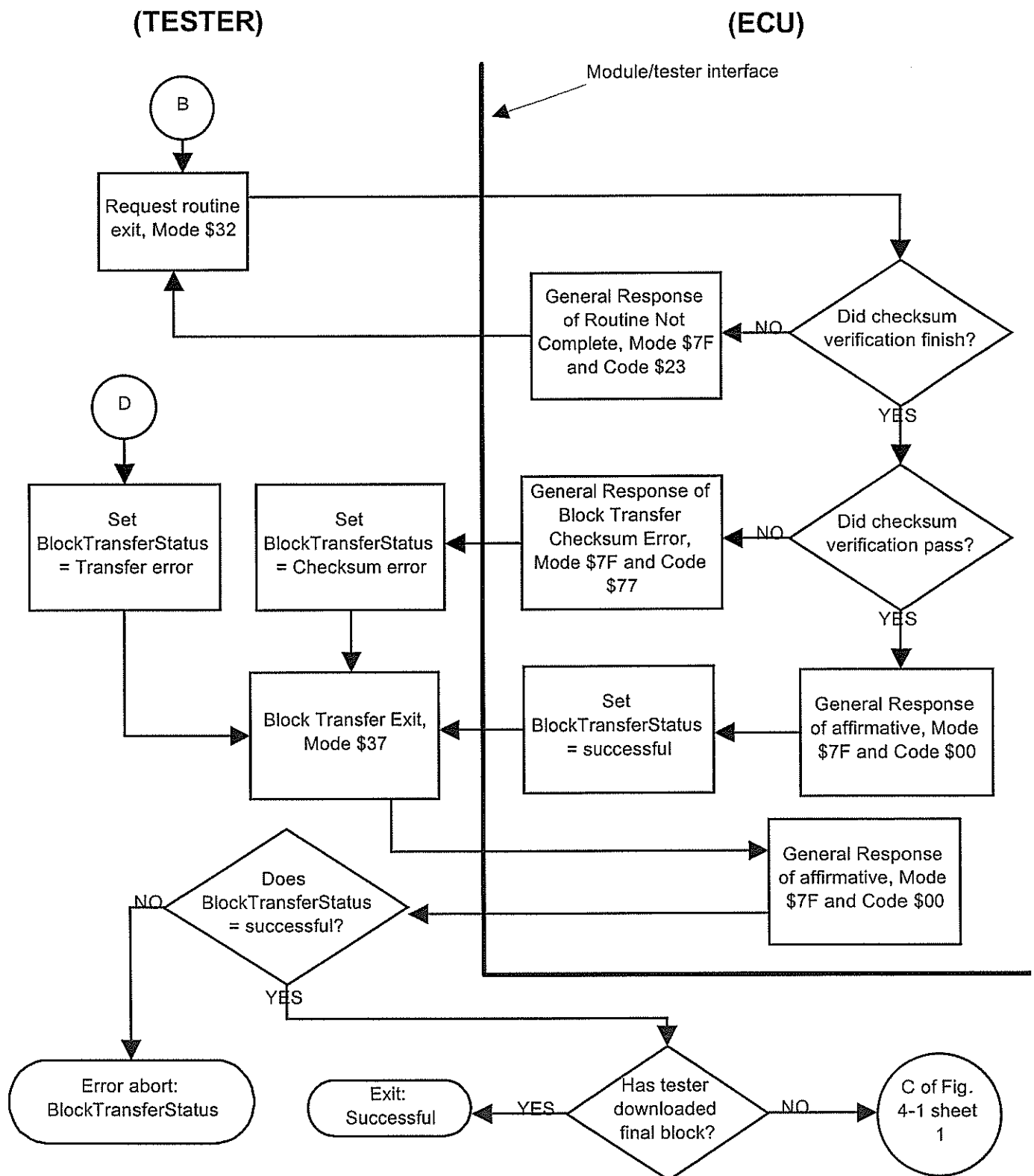


Figure 4-1 Method 3 Download Procedure with and without handshake (Sheet 3 of 3)

2.2.4 Method 4 FLASH Download

FLASH download is performed on modules that require programming of FLASH memory. A separate flowchart and procedure is characterized differently from the previously defined Method 3 procedure due to inherent differences in how the memory is operated upon.

Fig. 4-2 details the Method 4 procedure. It begins by making sure the file checksum provided in the header matches the file checksum calculated by the tester. It then proceeds to erase the entire FLASH memory area as needed for EEC-V and PTEC FLASH memory, erasing blocks of memory bytes is the only erase operation available, for example, options to erase only block 1 exist. Next, the setup and transfer of data is initiated by the tester. After the block is received by the module, a block checksum verification routine is entered to validate that the data was transferred correctly from the tester to the ECU (NOTE: the last two data bytes of the last block message contains the block checksum). After all of the blocks in the file are transferred to the module, a complete file checksum verification is performed on the portion of the FLASH memory that was programmed. If the checksum is successful then a proper download of the file configuration information has been executed; otherwise, there was some error and the process should be started over again from the beginning.

FLASH programming has only been implemented and targeted for Powertrain Control Modules when this Method 4 specification was written; however, any electronic control module performing any type of FLASH memory transfers can utilize this procedure. Also, note that Security access is optional for FLASH and shall be listed appropriately in it's corresponding Part II/SSDS.

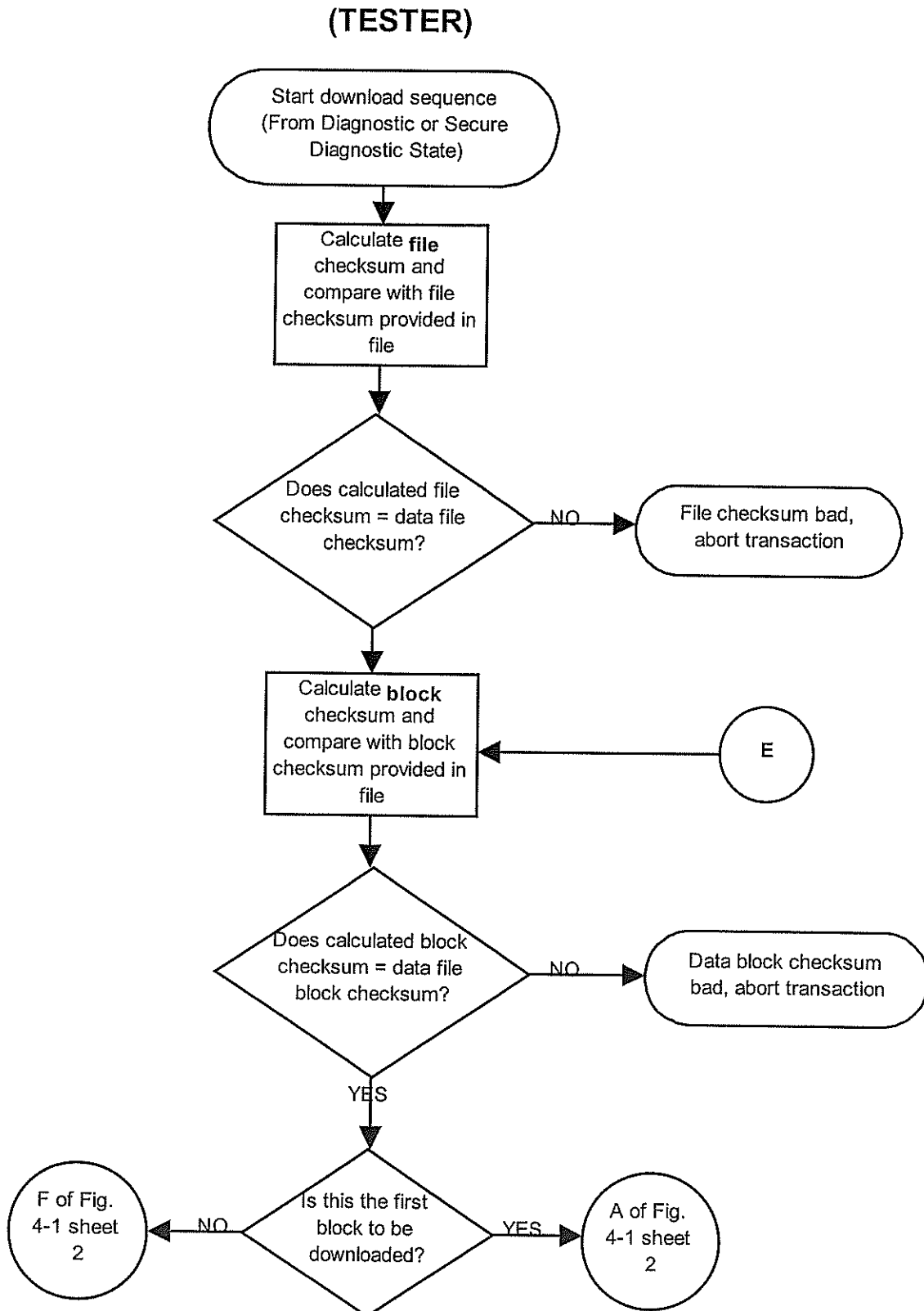


Figure 4-2 Method 4 IVFER Download Procedure (Sheet 1 of 4)

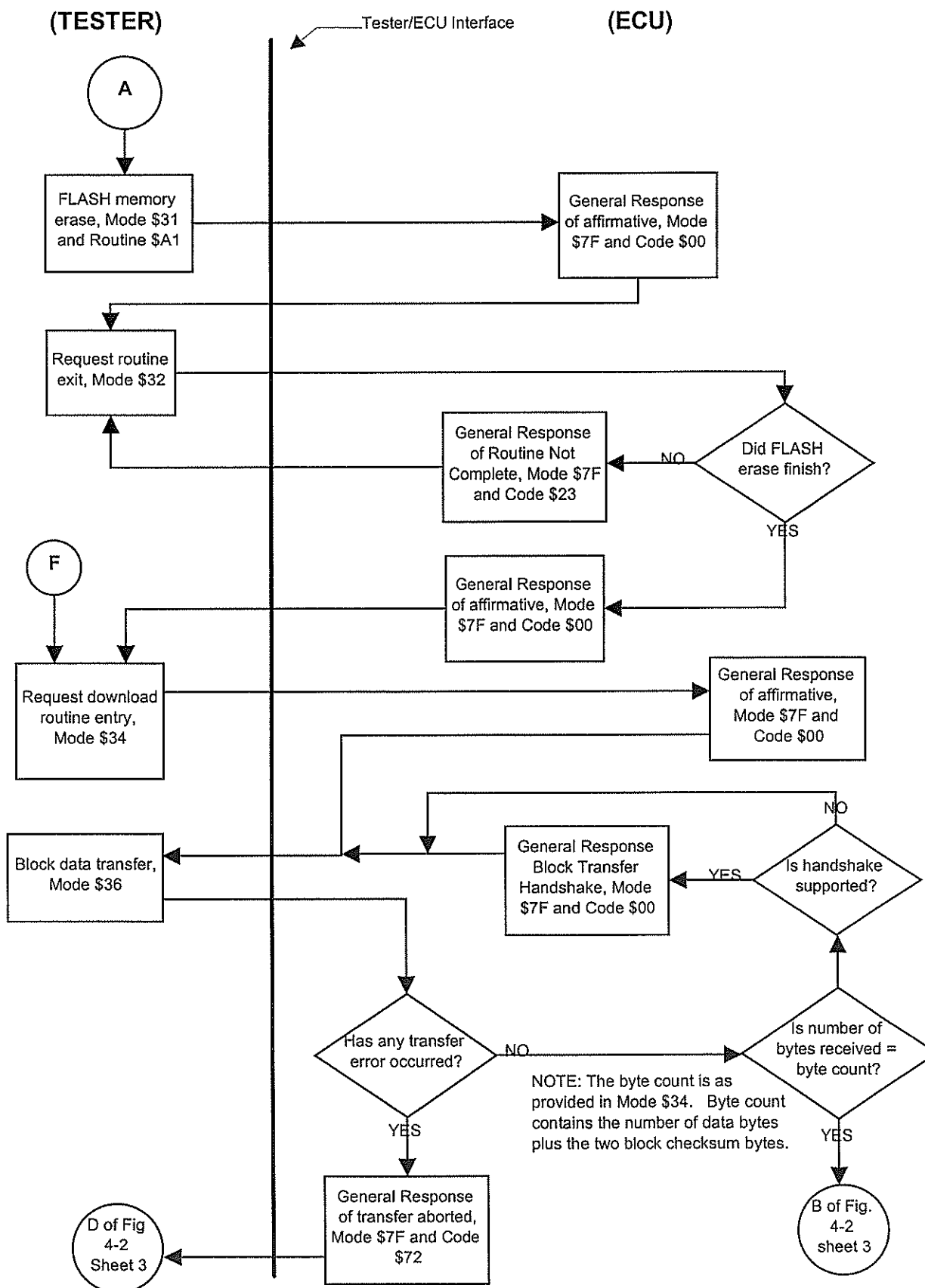


Figure 4-2 Method 4 IVER Download Procedure (Sheet 2 of 4)

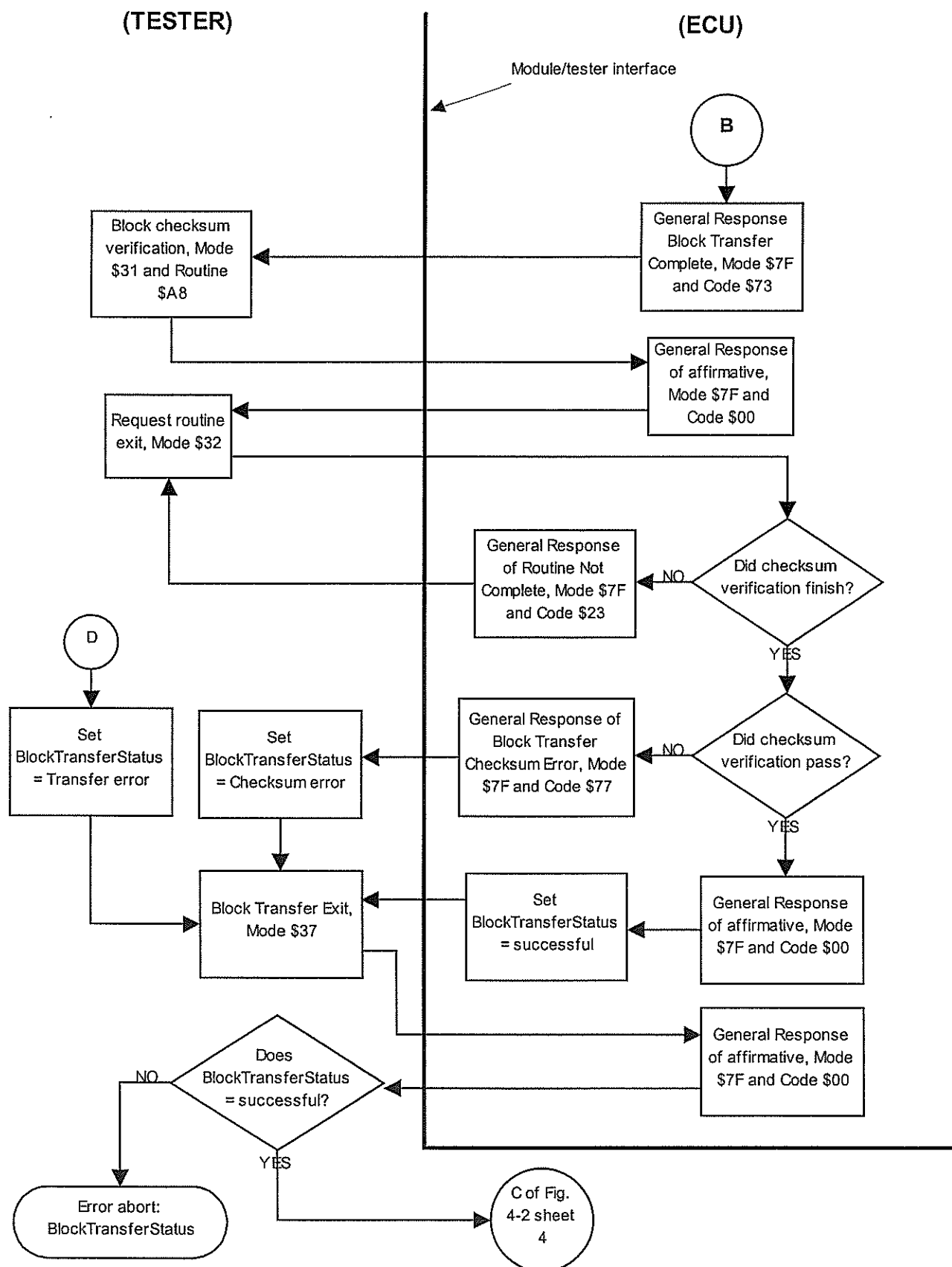


Figure 4-2 Method 4 IVFER Download Procedure (Sheet 3 of 4)

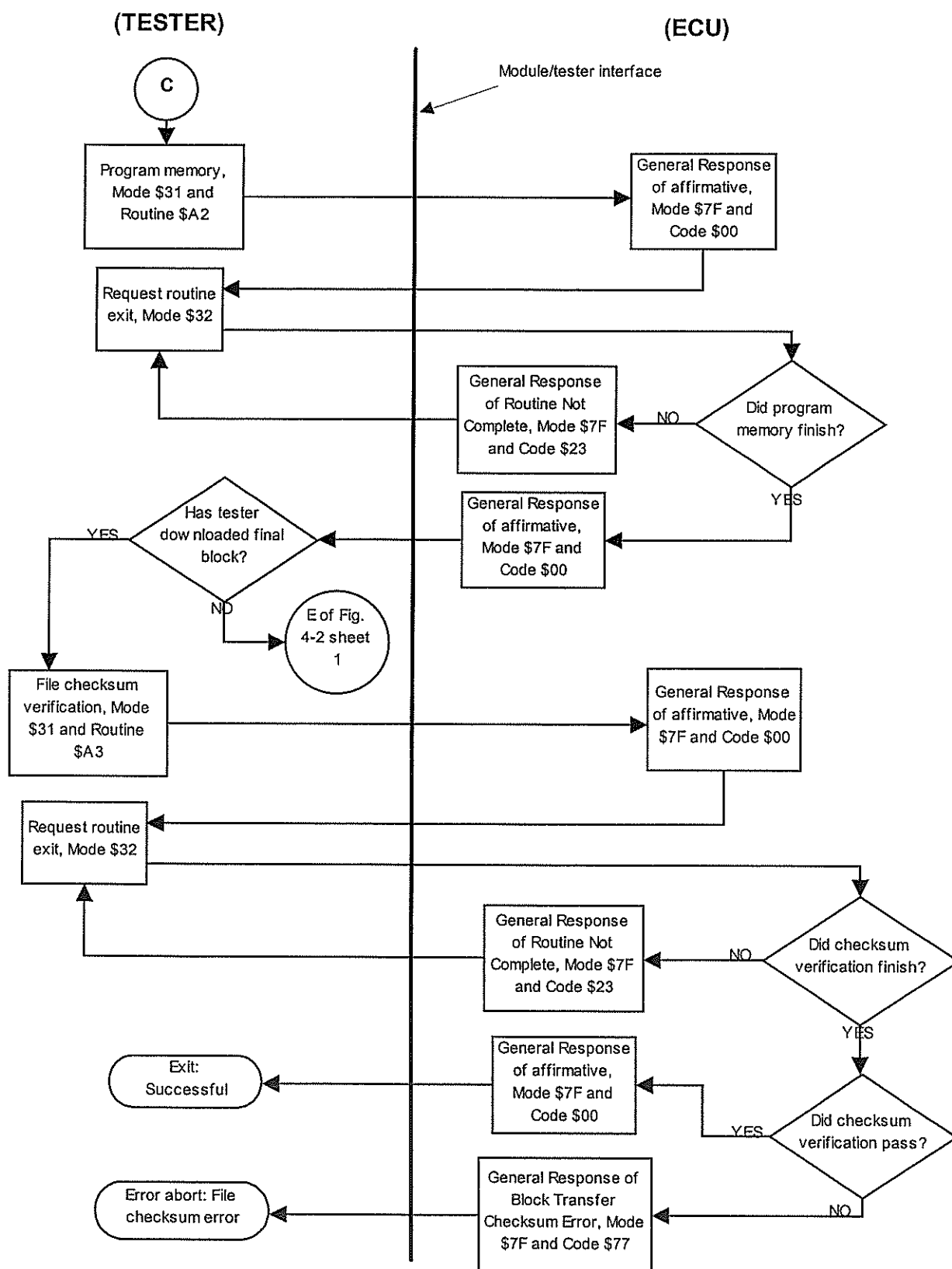


Figure 4-2 Method 4 IVFER Download Procedure (Sheet 4 of 4)

2.3 Unsupported Method 1 Configuration

Method 1 configuration isn't supported for any modules signing up to Version B or later of the Module Programming/Configuration Design Specification. It involved using **Diagnostic Command** (mode \$B1) to write information to modules. Command number \$0155 was the command issued to modules that supported this method. PID number \$C9FE was used to read the latest configuration residing in the module. It has been replaced with Method 2 as described above.

2.4 Base Strategy

Independent of what method the ECU is programmed with, each module should contain enough of a base or non-configured strategy to provide basic module functionality. It is important for modules to be shipped to assembly plants with a level of configuration such that certain tests can be performed before configuration is performed. For example, if a module is tested in trim before configuration occurs at EOL, the features tested must be configured in the module before arrival to the assembly facility.

If two configurable options cannot be enabled together, then the high volume option is to be enabled; for example, Daytime Running Lights should be default enabled for vehicles primarily shipped to regions with legislation mandating DRL. If there is question as to whether a function is considered critical to assembly, please contact the EESE Diagnostics sections for recommendations.

2.5 Programming Error Detection, Verification and Visual Indicators

All non-Ford powertrain defined modules received in assembly or service are to have continuous code DTC \$A477 logged as well as setting an erase inhibitor for \$A477; the erase inhibitor, when set, doesn't allow a tester to clear \$A477. \$A477 stands for "**Module Configuration Failure**" and is to be set for new modules that haven't been configured and only have a base strategy, or if a configuration problem occurs. \$A477 informs assembly and service test and configuration equipment that configuration needs to be performed, or the vehicle may not be safe or have all supported feature content operational. Modules are to write a Diagnostic Trouble Code (DTC) to report an invalid configuration to alert assembly plants and dealerships of a module requiring configuration.

Resetting the \$A477 erase inhibitor is to occur only when a configuration is sent to the module from a tester and the module accepts the configuration. After the erase inhibitor is reset, a tester may erase the \$A477; otherwise, a tester cannot erase \$A477 until the erase inhibitor is reset. The recommended strategy for testers, is to erase \$A477 only after a configuration has been successfully downloaded to the module, and it has been formally verified that the contents sent to the module from the tester made it as described below.

Programmed modules are to be verified for correct configuration before vehicles are shipped. Each method has a formal process for verifying that the module was programmed. In the case of critical functions or safety related functions, each D&R activity shall put checks within the module for invalid programming configurations or unprogrammed scenarios.

Wherever possible, the use of visual indication to alert plant and service personnel that there is a configuration problem (unconfigured or configured incorrectly) is strongly advised. Visual indication will prevent the majority of vehicles from reaching the customer with configuration failures. The following are the verification methods for the four types of programming:

1) Method 2 Download Verification: On-board and off-board verification schemes will be employed to verify correct configuration/programming. The on-board ECU will verify that the information that is programmed is part of the acceptable set of combinations allowed for safety critical modules, optional for non-safety related modules. Modules that detect an incorrect configuration are to log DTC \$A477 and/or flash indicators to visually convey a configuration problem. Aside from the ECU performing internal configuration checks, the off-board tester will perform the appropriate number of **Request Read Memory Block** (mode \$3C) messages to validate that what is in the module is the same as what was downloaded from the tester.

2) Method 3 and 4 Download Verification Method: The verification for these download methods is built in to the download procedures (Figs. 4-1 and 4-2).

Once the entire block of data has been downloaded, the tester initiates a block checksum in the module. A comparison of the downloaded block checksum and the ECU calculated checksum is performed by the module to verify the transfer was successful. Also, the ECU can check the data as it is received by the tester, as long as it meets handshaking timing requirements, and report an error during transfer to save the time of receiving the entire block for an error condition; this cannot be performed when the ECU communicates with Ford-9141 and no handshaking is the transfer format.

2.6 Information Back-up

A strong need exists for transferring configuration and calibration information to dealerships worldwide. Module programming/configuration complicates the amount of information needed to repair the vehicle. To aid in minimizing the amount of information needed to reprogram a module, there is a provision for an on-vehicle back-up of vehicle configuration that can be accessed by service test tools. The on-vehicle configuration data back-up resides in the PCM memory in the Vehicle ID (VID) Block and will also reside in a corporate, as-built, configuration database. The VID block contains the Vehicle Identification Number (VIN), powertrain and body/chassis/electrical configuration data and the vehicle module content. VID block data is reserved for bitmapped (e.g., enabled/disabled or present/not present) and other types of data that use small amounts of VID block memory (ex., tire/axle ratio, tire size). The VID block is assumed to be programmed whenever the delimiter character "*" in byte 22 of the VID block is present.

3. Service Requirements

3.1 Overview

The service center, dealership or independent, must have the capability and process for performing module programming/configuration. The service center will be required to do the following:

- Modify current module configuration - customer request feature change/upgrade. Legislated requirements that can be configured differently for various regions aren't to be modified per customer request and always need to follow local regional laws and regulations.
- Configure generic modules - module replaced with generic unconfigured modules
- Flash PCMs - update calibration or replace old strategy/calibration
- Access Vehicle ID Block for vehicle configuration - determine modules/features on vehicle
- Order configured parts from part depot/distribution center
- Remotely access as-built configuration/programming information

These requirements will need to be supported worldwide either through use of Ford approved service tools or through ordering configured parts through part depots/distribution centers.

3.2 Service Procedures for Module Programming/Configuration

3.2.1 Inhale/Exhale Method of Module Configuration

Providing a configuration process that attempts to put most of the burden on the tester, without having a tester resident database, to configure service modules is the intent of inhale/exhale process. See Fig. 5-1 for a flowchart characterizing the inhale/exhale procedure. As soon as the tester knows that a module containing configuration information is to be serviced with a new module it attempts to get all of the configuration information from the vehicle so that the technician doesn't need to access a remote source for the configuration information or know certain vehicle features. Having configuration information in the VID block allows most service configurations to happen electronically with the use of the mandatory electronic service tool and leaves remote access to the Hotline support group (**may change**) as a last resort.

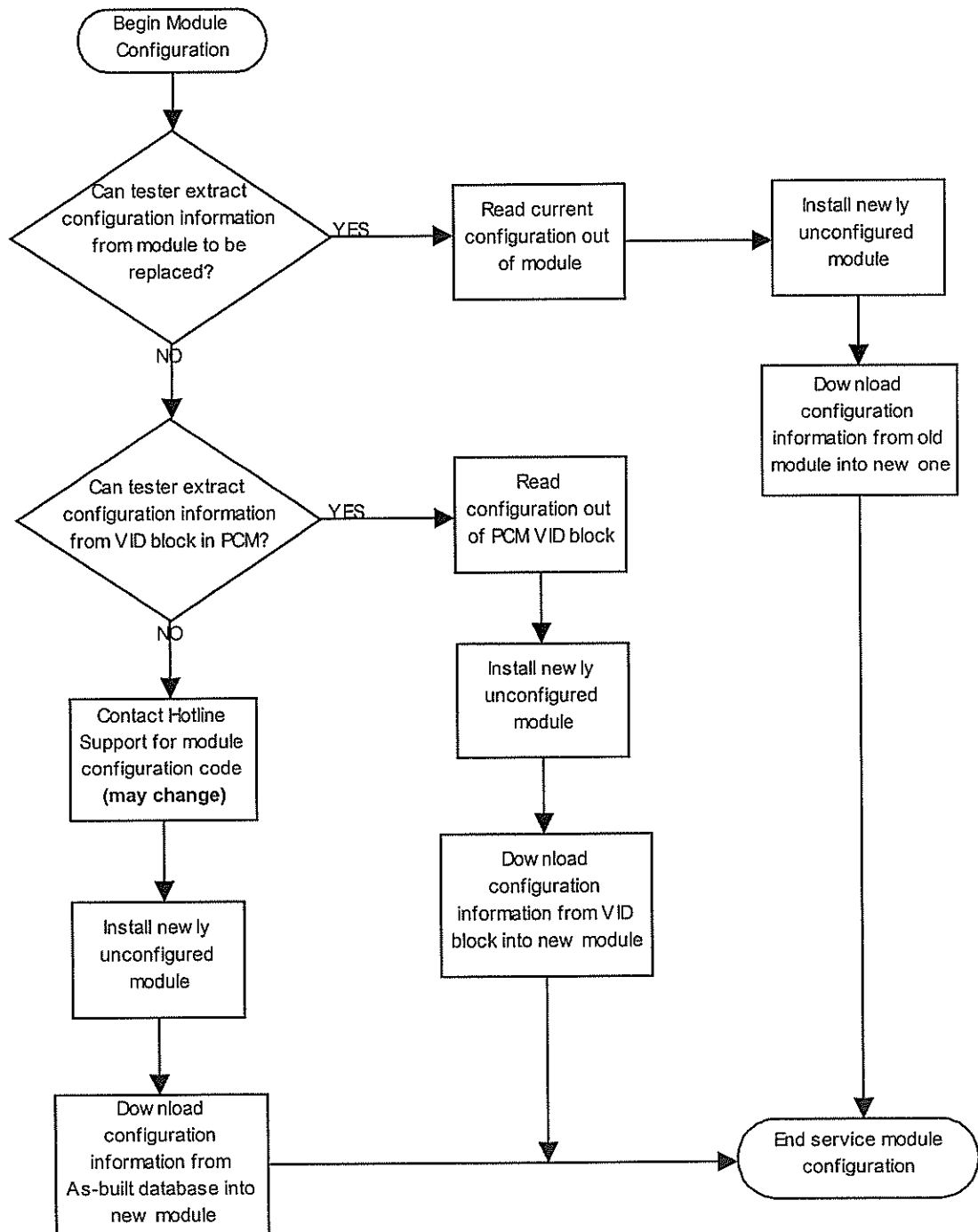


Fig. 5-1 Module Configuration Process for Service

3.2.2 On-line database access

This future TBD on-line database will, at a minimum, track original programming/configuration of all ECUs from assembly plant configuration data. It is currently planned that the Hotline Support activity will access this database for service centers needing information from it.

4. References

4.1 Ford Specifications

- ES_F7LC-1A294-AA "Generic Diagnostics Specification, (Part 1) Worldwide Requirements for ECU Subsystems"

4.2 SAE Specifications

- J1978 OBD Scan Tool
- J1979 E/E Diagnostic Test Modes
- J2186 Security Access
- J2190 Enhanced Diagnostic Test Modes